

## 10.2 The Cholesky factorization

If the cross-product matrix is to be used at all, the normal equations should be solved by factoring it as a triangular matrix times its transpose. One factorization of the augmented cross-product matrix then delivers every standard regression quantity.

### 10.2.1 The algorithm

Chapter 1 proved that a positive definite matrix has a unique factorization  $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$  with  $\mathbf{L}$  lower triangular and positive on the diagonal (Theorem 1.7.4(d)). Numerical work usually writes it with the upper triangular factor  $\mathbf{R} = \mathbf{L}^\top$ .

#### Theorem 10.2.1 — Cholesky factorization

Let  $\mathbf{A}$  be a  $p \times p$  positive definite matrix.

- (a) There is exactly one upper triangular  $\mathbf{R}$  with positive diagonal entries such that  $\mathbf{A} = \mathbf{R}^\top \mathbf{R}$ .
- (b) Its rows are given, for  $j = 1, \dots, p$  in turn, by

$$\begin{aligned} r_{jj} &= \left( a_{jj} - \sum_{k < j} r_{kj}^2 \right)^{1/2}, \\ r_{ji} &= \frac{1}{r_{jj}} \left( a_{ji} - \sum_{k < j} r_{kj} r_{ki} \right), \quad i > j, \end{aligned} \tag{10.2.1}$$

and every radicand in it is positive.

- (c) The computation takes  $p^3/3 + O(p^2)$  flops.
- (d) If  $\mathbf{A} = \mathbf{X}^\top \mathbf{X}$  and  $\mathbf{X} = \mathbf{Q}\mathbf{R}_0$  is a QR factorization with  $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_p$  and  $\mathbf{R}_0$  upper triangular with positive diagonal, then  $\mathbf{R}_0 = \mathbf{R}$ .

*Proof.*

- (a) is Theorem 1.7.4(d) with  $\mathbf{R} = \mathbf{L}^\top$ .
- (b) Entry  $(j, i)$ ,  $i \geq j$ , of  $\mathbf{R}^\top \mathbf{R}$  is  $\sum_{k \leq j} r_{kj} r_{ki}$ , because  $r_{kj} = 0$  for  $k > j$ . Setting it equal to  $a_{ji}$  and isolating the  $k = j$  term gives the recursion Eq. 10.2.1. So the entries of the unique factor of (a) satisfy Eq. 10.2.1, and Eq. 10.2.1 determines row  $j$  from rows  $1, \dots, j-1$ . By induction on  $j$ , the rows computed by Eq. 10.2.1 are those of the factor in (a). In particular the  $j$ th radicand is  $r_{jj}^2 > 0$ .
- (c) Row  $j$  needs about  $2(j-1)$  flops for each of its  $p-j+1$  entries, so the total is  $\sum_j 2(j-1)(p-j+1) = p^3/3 + O(p^2)$ .
- (d)  $\mathbf{X}^\top \mathbf{X} = \mathbf{R}_0^\top \mathbf{Q}^\top \mathbf{Q} \mathbf{R}_0 = \mathbf{R}_0^\top \mathbf{R}_0$ , and uniqueness in (a).

■

By part (d), the triangle of a QR factorization of  $\mathbf{X}$  and the Cholesky factor of  $\mathbf{X}^\top \mathbf{X}$  are the same matrix (see also Exercise (B2)). The two routes of this chapter compute the same

object, one from  $\mathbf{X}$ , the other from  $\mathbf{X}^\top \mathbf{X}$  after the small singular values may already have been damaged (Proposition 10.1.3).

The algorithm itself is backward stable *for the factorization*. The computed  $\hat{\mathbf{R}}$  satisfies  $\hat{\mathbf{R}}^\top \hat{\mathbf{R}} = \mathbf{A} + \Delta \mathbf{A}$  with  $|\Delta \mathbf{A}| \leq \gamma_{p+1} |\hat{\mathbf{R}}|^\top |\hat{\mathbf{R}}|$  entrywise, and it runs to completion whenever  $\mathbf{A}$  is not too close to singular (Higham 2002, chapter 10), with no pivoting. The weakness of the normal equations lies entirely in forming  $\mathbf{A} = \mathbf{X}^\top \mathbf{X}$ , not in factoring it.

```
import numpy as np
import statsmodels.api as sm

def cholesky_upper(A):
    """Upper triangular R with positive diagonal and R'R = A (A positive definite)."""
    A = np.array(A, dtype=float)
    p = A.shape[0]
    R = np.zeros_like(A)
    for j in range(p):
        d = A[j, j] - R[:j, j] @ R[:j, j]
        if d <= 0:
            raise np.linalg.LinAlgError(f"not positive definite at step {j + 1}")
        R[j, j] = np.sqrt(d)
        R[j, j + 1:] = (A[j, j + 1:] - R[:j, j] @ R[:j, j + 1:]) / R[j, j]
    return R

def solve_upper(R, b):
    """Back substitution for R x = b, R upper triangular."""
    x = np.zeros(np.shape(b))
    for i in range(len(b) - 1, -1, -1):
        x[i] = (b[i] - R[i, i + 1:] @ x[i + 1:]) / R[i, i]
    return x

def solve_lower(L, b):
    """Forward substitution for L x = b, L lower triangular."""
    x = np.zeros(np.shape(b))
    for i in range(len(b)):
        x[i] = (b[i] - L[i, :i] @ x[:i]) / L[i, i]
    return x
```

[Run this code online](#)

## 10.2.2 Everything from one factorization

Solving the normal equations  $\mathbf{R}^\top \mathbf{R} \mathbf{b} = \mathbf{X}^\top \mathbf{y}$  takes two triangular solves:  $\mathbf{R}^\top \mathbf{z} = \mathbf{X}^\top \mathbf{y}$  for  $\mathbf{z}$ , then  $\mathbf{R} \mathbf{b} = \mathbf{z}$ . Each costs  $p^2$  flops. It is neater to factor the cross-product matrix of the augmented matrix  $\mathbf{Z} = [\mathbf{X}, \mathbf{y}]$ . The factor then contains the solution of the first triangular system and the residual sum of squares.

**Proposition 10.2.2 — The augmented factorization**

Let  $\mathbf{X}$  be  $n \times p$  of full column rank, suppose  $\mathbf{y} \notin \mathcal{C}(\mathbf{X})$ , and let

$$\mathbf{Z}^\top \mathbf{Z} = \begin{pmatrix} \mathbf{X}^\top \mathbf{X} & \mathbf{X}^\top \mathbf{y} \\ \mathbf{y}^\top \mathbf{X} & \mathbf{y}^\top \mathbf{y} \end{pmatrix} = \mathbf{T}^\top \mathbf{T}, \quad \mathbf{T} = \begin{pmatrix} \mathbf{R} & \mathbf{z} \\ \mathbf{0}^\top & d \end{pmatrix},$$

be the Cholesky factorization (Theorem 10.2.1) of the cross-product matrix of  $\mathbf{Z} = [\mathbf{X}, \mathbf{y}]$ . Then:

- (a)  $\mathbf{R}$  is the Cholesky factor of  $\mathbf{X}^\top \mathbf{X}$ , the least squares estimate solves  $\mathbf{R}\hat{\boldsymbol{\beta}} = \mathbf{z}$ , and  $d^2 = \text{SSE} = \|\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|^2$ ;
- (b) for each  $k \leq p$ , the residual sum of squares of the regression of  $\mathbf{y}$  on the first  $k$  columns of  $\mathbf{X}$  is  $d^2 + \sum_{j>k} z_j^2$ . The drop in residual sum of squares when column  $k$  joins columns  $1, \dots, k-1$  is  $z_k^2$ ;
- (c)  $\det(\mathbf{X}^\top \mathbf{X}) = \prod_j r_{jj}^2$ ,  $(\mathbf{X}^\top \mathbf{X})^{-1} = \mathbf{R}^{-1} \mathbf{R}^{-\top}$ , and the leverage of row  $\mathbf{x}_{(i)}^\top$  is  $h_{ii} = \|\mathbf{R}^{-\top} \mathbf{x}_{(i)}\|^2$ .

*Proof.*  $\mathbf{Z}$  has full column rank because  $\mathbf{y} \notin \mathcal{C}(\mathbf{X})$ , so  $\mathbf{Z}^\top \mathbf{Z}$  is positive definite and  $\mathbf{T}$  exists. Multiplying out  $\mathbf{T}^\top \mathbf{T}$  and comparing blocks with  $\mathbf{Z}^\top \mathbf{Z}$  gives

$$\mathbf{R}^\top \mathbf{R} = \mathbf{X}^\top \mathbf{X}, \quad \mathbf{R}^\top \mathbf{z} = \mathbf{X}^\top \mathbf{y}, \quad \mathbf{z}^\top \mathbf{z} + d^2 = \mathbf{y}^\top \mathbf{y}.$$

The first identity and uniqueness make  $\mathbf{R}$  the Cholesky factor of  $\mathbf{X}^\top \mathbf{X}$ . The second gives  $\mathbf{R}^\top \mathbf{R}\hat{\boldsymbol{\beta}} = \mathbf{R}^\top \mathbf{z}$  for the solution of the normal equations, so  $\mathbf{R}\hat{\boldsymbol{\beta}} = \mathbf{z}$ .

For (b), and for the rest of (a), let  $\mathbf{Q} = \mathbf{Z}\mathbf{T}^{-1}$ . Then  $\mathbf{Q}^\top \mathbf{Q} = \mathbf{T}^{-\top} \mathbf{Z}^\top \mathbf{Z} \mathbf{T}^{-1} = \mathbf{I}$ , so the columns  $\mathbf{q}_1, \dots, \mathbf{q}_{p+1}$  of  $\mathbf{Q}$  are orthonormal, and  $\mathbf{Z} = \mathbf{Q}\mathbf{T}$ . Because  $\mathbf{T}$  is upper triangular with nonzero diagonal, the first  $k$  columns of  $\mathbf{X}$  span the same subspace as  $\mathbf{q}_1, \dots, \mathbf{q}_k$ . The last column of  $\mathbf{Z} = \mathbf{Q}\mathbf{T}$  reads  $\mathbf{y} = \sum_{j \leq p} z_j \mathbf{q}_j + d \mathbf{q}_{p+1}$ . By Proposition 6.3.2 the projection of  $\mathbf{y}$  onto the span of the first  $k$  columns is  $\sum_{j \leq k} z_j \mathbf{q}_j$ , and the residual sum of squares is  $\|\mathbf{y}\|^2 - \sum_{j \leq k} z_j^2 = d^2 + \sum_{j>k} z_j^2$ . With  $k = p$  this is  $d^2$ , which completes (a). Taking differences gives the rest of (b).

- (c) The determinant of a triangular matrix is the product of its diagonal. The inverse formula follows from  $\mathbf{X}^\top \mathbf{X} = \mathbf{R}^\top \mathbf{R}$ . Finally  $h_{ii} = \mathbf{x}_{(i)}^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_{(i)} = \mathbf{x}_{(i)}^\top \mathbf{R}^{-1} \mathbf{R}^{-\top} \mathbf{x}_{(i)} = \|\mathbf{R}^{-\top} \mathbf{x}_{(i)}\|^2$ .

■

Part (b) says that the squares  $z_1^2, \dots, z_p^2$  are the *sequential sums of squares* of the regressors in the order of the columns (Definition 9.3.1). A single factorization gives the whole nested sequence of fits. This is the content of Theorem 6.5.1, now in coordinates. Part (c) costs about  $np^2$  flops for all  $n$  leverages, each a triangular solve of  $p^2$  flops.

### Example. (Murder rates by the augmented factorization)

For the state data of Example (Poverty and murder rates) ( $n = 50$ , with an intercept and the percentages in poverty, in single-parent households and urban, so  $p = 4$ ), the augmented factorization (Proposition 10.2.2) gives the coefficients  $-9.0946, 0.2538, 0.3980$  and  $0.0046$ , and the corner entry  $d = 10.0976$ , so  $\text{SSE} = d^2 = 101.96$ . The condition number of  $\mathbf{X}$  is only 509, and the solution agrees with a QR solution to a

relative difference of  $2 \times 10^{-14}$ . The largest leverage is 0.260, and  $\log_{10} \det(\mathbf{X}^\top \mathbf{X}) = 11.14$ . The script also checks the standard errors  $s \|e_j^\top \mathbf{R}^{-1}\|$  against statsmodels.

```
data = sm.datasets.statcrime.load_pandas().data.drop(index="District of Columbia")
y = data["murder"].to_numpy()
X = np.column_stack([np.ones(len(y)), data[["poverty", "single", "urban"]]])
n, p = X.shape
```

[Run this code online](#)

```
Z = np.column_stack([X, y])          # augmented matrix [X, y]
S = Z.T @ Z                          # (p+1) x (p+1) cross-product matrix
T = cholesky_upper(S)                 # T = [[R, z], [0, d]]
R, z, d = T[:p, :p], T[p, p], T[p, p]

beta = solve_upper(R, z)              # R b = z
sse = d ** 2                          # the last diagonal entry is sqrt(SSE)
K = solve_upper(R, np.eye(p))         # R^{-1}
XtX_inv = K @ K.T                     # (X'X)^{-1} = R^{-1} R^{-T}
h = np.array([np.sum(solve_lower(R.T, x) ** 2) for x in X]) # leverages
print("coefficients:", np.round(beta, 4))
print(f"SSE = {sse:.4f}, d = {d:.4f}")
```

[Run this code online](#)

### 10.2.3 When the normal equations are good enough

Proposition 10.1.3 and Eq. 10.1.2 predict a relative error of order  $\kappa^2 u$  in  $\hat{\beta}$  from the normal equations, where, as Section 10.5 shows,  $\kappa$  is the condition number of  $\mathbf{X}$  after its columns are scaled to equal length. With  $\kappa \leq 10^4$  about eight digits survive, more than any coefficient is known statistically. The normal equations are therefore sound for well-conditioned designs, when only cross-products are available, and inside iterations such as the reweighted least squares of Chapter 34. Nearly collinear observational data are where the  $\kappa^2$  loss bites.

### 10.2.4 The sweep operator

Before QR became standard, regression programs worked on the augmented cross-product matrix with an operation that adds one variable at a time to the fit. It is still used for fitting many subsets of variables.

#### Definition 10.2.1 — Sweep

Let  $\mathbf{A}$  be symmetric with  $a_{kk} \neq 0$ . **Sweeping**  $\mathbf{A}$  on pivot  $k$  produces the symmetric matrix  $\mathbf{B}$  with

$$b_{kk} = -\frac{1}{a_{kk}}, \quad b_{ik} = b_{ki} = \frac{a_{ik}}{a_{kk}}, \quad b_{ij} = a_{ij} - \frac{a_{ik}a_{kj}}{a_{kk}} \quad (i, j \neq k).$$

### Proposition 10.2.3 — What sweeping computes

Let  $\mathbf{S} = \mathbf{Z}^\top \mathbf{Z}$  be the cross-product matrix of  $\mathbf{Z} = [\mathbf{z}_1, \dots, \mathbf{z}_m]$ . Let  $K$  be a set of column indices such that  $\mathbf{Z}_K$  has full column rank, and let  $L$  be the remaining indices. Sweeping  $\mathbf{S}$  on the pivots in  $K$ , one at a time and in any order, meets only positive pivots and produces the matrix  $\mathbf{W}$  with blocks

$$\mathbf{W}_{KK} = -(\mathbf{Z}_K^\top \mathbf{Z}_K)^{-1}, \quad \mathbf{W}_{KL} = \mathbf{W}_{LK}^\top = (\mathbf{Z}_K^\top \mathbf{Z}_K)^{-1} \mathbf{Z}_K^\top \mathbf{Z}_L, \quad \mathbf{W}_{LL} = \mathbf{Z}_L^\top (\mathbf{I} - \mathbf{M}_K) \mathbf{Z}_L,$$

where  $\mathbf{M}_K$  is the projection onto  $\mathcal{C}(\mathbf{Z}_K)$ . Column  $j \in L$  of  $\mathbf{W}_{KL}$  holds the least squares coefficients of  $\mathbf{z}_j$  regressed on  $\mathbf{Z}_K$ , and  $\mathbf{W}_{LL}$  holds the residual cross-products. With  $\mathbf{Z} = [\mathbf{X}, \mathbf{y}]$  and  $K$  the columns of  $\mathbf{X}$ , the last column of  $\mathbf{W}$  contains  $\hat{\beta}$  and, in its corner, the residual sum of squares.

*Proof.* We induct on the size of  $K$ ; the case  $K = \emptyset$  is  $\mathbf{W} = \mathbf{S}$ . Add a pivot  $m \in L$  with  $\mathbf{Z}_{K \cup \{m\}}$  of full column rank, and write  $\mathbf{G} = (\mathbf{Z}_K^\top \mathbf{Z}_K)^{-1}$ ,  $\mathbf{B}_j = \mathbf{G} \mathbf{Z}_K^\top \mathbf{z}_j$  and  $\mathbf{w} = (\mathbf{I} - \mathbf{M}_K) \mathbf{z}_m$ . The pivot is  $\mathbf{z}_m^\top (\mathbf{I} - \mathbf{M}_K) \mathbf{z}_m = \|\mathbf{w}\|^2 > 0$ . For  $i, j \in L \setminus \{m\}$  the new entries are  $\mathbf{z}_i^\top (\mathbf{I} - \mathbf{M}_K - \mathbf{w} \mathbf{w}^\top / \|\mathbf{w}\|^2) \mathbf{z}_j = \mathbf{z}_i^\top (\mathbf{I} - \mathbf{M}_{K \cup \{m\}}) \mathbf{z}_j$  by Lemma 6.6.1. The new entries  $\mathbf{w}^\top \mathbf{z}_j / \|\mathbf{w}\|^2$  in row  $m$ , and  $\mathbf{B}_j - \mathbf{B}_m \mathbf{w}^\top \mathbf{z}_j / \|\mathbf{w}\|^2$  in the rows of  $K$ , are the coefficients of  $\mathbf{z}_j$  on  $[\mathbf{Z}_K, \mathbf{z}_m]$  by Theorem 6.6.2. The partitioned inverse (Theorem 1.6.4), with Schur complement  $\|\mathbf{w}\|^2$ , shows that  $(\mathbf{Z}_{K \cup \{m\}}^\top \mathbf{Z}_{K \cup \{m\}})^{-1}$  has blocks  $\mathbf{G} + \mathbf{B}_m \mathbf{B}_m^\top / \|\mathbf{w}\|^2$ ,  $-\mathbf{B}_m / \|\mathbf{w}\|^2$  and  $1 / \|\mathbf{w}\|^2$ , whose negatives are what the sweep produces:  $-\mathbf{G} - \mathbf{B}_m \mathbf{B}_m^\top / \|\mathbf{w}\|^2$ ,  $\mathbf{B}_m / \|\mathbf{w}\|^2$  and  $-1 / \|\mathbf{w}\|^2$ . The result depends only on the set  $K$ , so the order of the sweeps does not matter. ■

Sweeping a pivot costs about  $(p+1)^2$  flops, so a new variable enters a fitted model for  $O(p^2)$  work. The price is the same as for the normal equations: the sweep works with cross-products and inherits their  $\kappa^2$  sensitivity. For the state data, sweeping all four columns reproduces the coefficients and SSE of [Example \(Murder rates by the augmented factorization\)](#). Sweeping only the intercept and the single-parent column puts the SSE of that two-column model, 121.92, in the corner.

```
def sweep(A, k):
    """Sweep the symmetric matrix A on pivot k."""
    A = np.array(A, dtype=float)
    a = A[k, k]
    B = A - np.outer(A[:, k], A[k, :]) / a
    B[k, :] = A[k, :] / a
    B[:, k] = A[:, k] / a
    B[k, k] = -1 / a
    return B

W = S.copy()
for k in range(p):
    W = sweep(W, k)
print("coefficients from the sweep:", np.round(W[:p, p], 4))
print(f"SSE from the sweep: {W[p, p]:.4f}")
```

[Run this code online](#)

## 10.2.5 Forming cross-products: the centring problem

With an intercept in the model, the cross-product matrix is usually computed in centred form,  $\sum_i (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$ , which by Theorem 6.7.1 changes the parameterization but not the fit. How it is computed matters a great deal. The “textbook” formula  $\sum_i x_i^2 - n\bar{x}^2$  subtracts two large, nearly equal numbers whenever the mean is large compared with the spread. The loss is governed by the ratio

$$\kappa_{\mathbf{x}} = \frac{\|\mathbf{x}\|}{\|\mathbf{x} - \bar{x}\mathbf{1}\|} = \left(1 + \frac{n\bar{x}^2}{\sum_i (x_i - \bar{x})^2}\right)^{1/2},$$

which is the condition number of the column, and it is large when the coefficient of variation is small. Three algorithms are in common use:

- *textbook*: accumulate  $\sum x_i$  and  $\sum x_i^2$  in one pass, then subtract;
- *two-pass*: compute  $\bar{x}$  in a first pass, then sum  $(x_i - \bar{x})^2$  in a second;
- *updating*: one pass, maintaining the running mean and the running centred sum by the recursions below.

### Proposition 10.2.4 — Updating means and centred cross-products

For vectors  $\mathbf{x}_1, \mathbf{x}_2, \dots$  in  $\mathbb{R}^q$  let  $\mathbf{m}_k$  be the mean of the first  $k$  and  $\mathbf{C}_k = \sum_{i \leq k} (\mathbf{x}_i - \mathbf{m}_k)(\mathbf{x}_i - \mathbf{m}_k)^\top$ . With  $\mathbf{d}_k = \mathbf{x}_k - \mathbf{m}_{k-1}$ ,

$$\mathbf{m}_k = \mathbf{m}_{k-1} + \frac{\mathbf{d}_k}{k}, \quad \mathbf{C}_k = \mathbf{C}_{k-1} + \frac{k-1}{k} \mathbf{d}_k \mathbf{d}_k^\top.$$

*Proof.* The first identity is  $k\mathbf{m}_k = (k-1)\mathbf{m}_{k-1} + \mathbf{x}_k$ . For the second, write  $\mathbf{C}_k = \sum_{i \leq k} \mathbf{x}_i \mathbf{x}_i^\top - k\mathbf{m}_k \mathbf{m}_k^\top$  and put  $\mathbf{m} = \mathbf{m}_{k-1}$ ,  $\mathbf{d} = \mathbf{d}_k$ . Then

$$\mathbf{C}_k - \mathbf{C}_{k-1} = \mathbf{x}_k \mathbf{x}_k^\top - k\mathbf{m}_k \mathbf{m}_k^\top + (k-1)\mathbf{m} \mathbf{m}^\top.$$

Substituting  $\mathbf{x}_k = \mathbf{m} + \mathbf{d}$  and  $k\mathbf{m}_k \mathbf{m}_k^\top = k\mathbf{m} \mathbf{m}^\top + \mathbf{m} \mathbf{d}^\top + \mathbf{d} \mathbf{m}^\top + \mathbf{d} \mathbf{d}^\top / k$ , all the terms in  $\mathbf{m}$  cancel and  $\mathbf{d} \mathbf{d}^\top (1 - 1/k)$  remains. ■

The update adds a *centred* rank-one term, so it never subtracts large quantities. It is the cross-product analogue of the row updates of Section 10.6. Chan, Golub and LeVeque (1983) give error bounds of roughly  $n\kappa_{\mathbf{x}}^2 u$  for the textbook formula,  $n\kappa_{\mathbf{x}} u$  for updating and  $nu + n^2 \kappa_{\mathbf{x}}^2 u^2$  for the two-pass algorithm.

### Example. (Centring a column with a large mean)

A sample of 1000 standard normal values is shifted by  $c = 1, 10, \dots, 10^8$ , and the centred sum of squares is computed by the three algorithms and compared with its exact value (found in rational arithmetic). At  $c = 10^6$ , where  $\kappa_{\mathbf{x}} = 1.0 \times 10^6$ , the relative errors are  $8.9 \times 10^{-5}$  for the textbook formula,  $8.1 \times 10^{-12}$  for updating and  $2.7 \times 10^{-17}$  for two passes. At  $c = 10^8$  the textbook formula has relative error 3.1: not a single correct

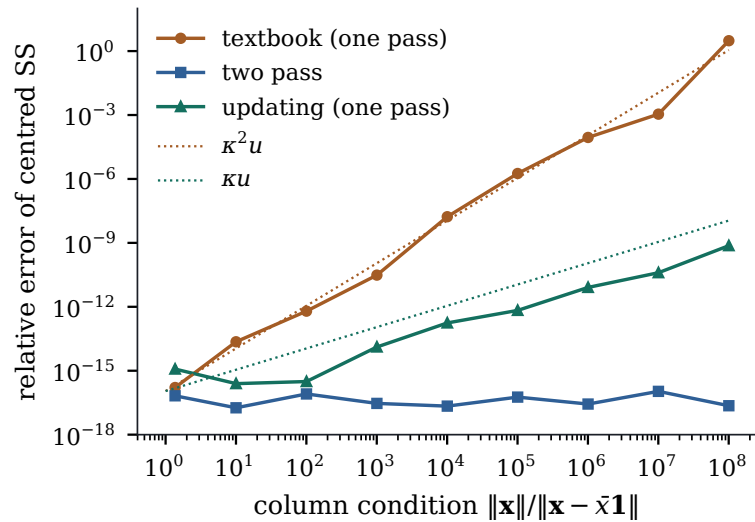


Figure 10.2.1: Relative error of the centred sum of squares of a column of 1000 values with standard deviation one and mean up to  $10^8$ , against the column condition number  $\kappa_x$ . The one-pass textbook formula loses accuracy like  $\kappa^2 u$ , the one-pass updating formula like  $\kappa u$ , and the two-pass algorithm not at all.

digit, and possibly a negative “sum of squares”. Updating gives  $7.4 \times 10^{-10}$  and two passes  $2.3 \times 10^{-17}$ . Figure 10.2.1 shows the three growth rates  $\kappa^2 u$ ,  $\kappa u$  and  $u$ .

```
def ss_textbook(x):
    return np.sum(x * x) - len(x) * np.mean(x) ** 2      # one pass, then subtract

def ss_two_pass(x):
    m = np.mean(x)                                       # first pass: the mean
    return np.sum((x - m) ** 2)                         # second pass: deviations

def ss_updating(x):
    m, s = 0.0, 0.0
    for k, xk in enumerate(x, start=1):                # one pass, no
        cancellation                                     # cancellation
        d = xk - m
        m = m + d / k
        s = s + (k - 1) / k * d * d
    return s
```

[Run this code online](#)

Calendar years, population totals and price indices have large means and small spread. A program that forms cross-products should centre them with two passes (or update), solve for the slopes, and recover the intercept as  $\bar{y} - \bar{x}^\top \hat{\beta}_{\text{slopes}}$ .

## 10.2.6 Exercises

### 10.2.6.1 A. Check your understanding

**Exercise A1.**

Compute the Cholesky factor of  $\mathbf{A} = \begin{pmatrix} 4 & 2 & -2 \\ 2 & 10 & 2 \\ -2 & 2 & 6 \end{pmatrix}$  by Eq. 10.2.1, and use it to solve  $\mathbf{A}\mathbf{b} = (2, 14, 6)^\top$  and to find  $\det \mathbf{A}$ .

*Solution.* Row 1:  $r_{11} = 2$ ,  $r_{12} = 1$ ,  $r_{13} = -1$ . Row 2:  $r_{22} = \sqrt{10-1} = 3$ ,  $r_{23} = (2 - (1)(-1))/3 = 1$ . Row 3:  $r_{33} = \sqrt{6-1-1} = 2$ . So  $\mathbf{R} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & 3 & 1 \\ 0 & 0 & 2 \end{pmatrix}$  and  $\det \mathbf{A} = (2 \cdot 3 \cdot 2)^2 = 144$ . Forward substitution in  $\mathbf{R}^\top \mathbf{z} = (2, 14, 6)^\top$  gives  $z_1 = 1$ ,  $z_2 = (14-1)/3 = 13/3$ ,  $z_3 = (6 + 1 - 13/3)/2 = 4/3$ . Back substitution in  $\mathbf{R}\mathbf{b} = \mathbf{z}$  gives  $b_3 = 2/3$ ,  $b_2 = (13/3 - 2/3)/3 = 11/9$  and  $b_1 = (1 - 11/9 + 2/3)/2 = 2/9$ . Check:  $\mathbf{A}\mathbf{b} = (8/9 + 22/9 - 12/9, 4/9 + 110/9 + 12/9, -4/9 + 22/9 + 36/9) = (2, 14, 6)$ .

**Exercise A2.**

For simple regression,  $\mathbf{X} = [\mathbf{1}, \mathbf{x}]$ , find the Cholesky factor (Theorem 10.2.1) of  $\mathbf{X}^\top \mathbf{X}$  and of the augmented matrix, and show that it reproduces the slope  $S_{xy}/S_{xx}$  of Theorem 5.2.1. Where does the centring happen?

*Solution.*  $\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} n & n\bar{x} \\ n\bar{x} & \sum x_i^2 \end{pmatrix}$  gives  $r_{11} = \sqrt{n}$ ,  $r_{12} = \sqrt{n}\bar{x}$  and  $r_{22} = (\sum x_i^2 - n\bar{x}^2)^{1/2} = S_{xx}^{1/2}$ . The augmented column gives  $z_1 = n\bar{y}/\sqrt{n} = \sqrt{n}\bar{y}$  and  $z_2 = (\sum x_i y_i - n\bar{x}\bar{y})/S_{xx}^{1/2} = S_{xy}/S_{xx}^{1/2}$ . Back substitution gives the slope  $b_2 = z_2/r_{22} = S_{xy}/S_{xx}$  and the intercept  $b_1 = (z_1 - r_{12}b_2)/r_{11} = \bar{y} - \bar{x}b_2$ . The centring happens in the subtraction  $a_{22} - r_{12}^2$ , which is the textbook formula of Example (Centring a column with a large mean). So the Cholesky factorization of an uncentred cross-product matrix inherits its cancellation.

**10.2.6.2 B. Practice****Exercise B1.**

Show that the standard error of  $\hat{\beta}_j$  is  $s \|e_j^\top \mathbf{R}^{-1}\|$ , where  $s^2 = \text{SSE}/(n-p)$  (Theorem 5.6.1). Show that all  $p$  standard errors cost  $p^3/3 + O(p^2)$  flops, and all  $n$  leverages  $np^2 + O(np)$  flops. Why is computing  $\mathbf{M}$  itself a bad idea?

*Solution.* By Theorem 5.5.1 the estimated variance of  $\hat{\beta}_j$  is  $s^2[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj}$ , and  $[(\mathbf{X}^\top \mathbf{X})^{-1}]_{jj} = e_j^\top \mathbf{R}^{-1} \mathbf{R}^{-\top} e_j = \|e_j^\top \mathbf{R}^{-1}\|^2$ . The inverse of a triangular matrix is triangular, and column  $j$  of  $\mathbf{R}^{-1}$  is found by back substitution on its leading  $j \times j$  block, at about  $j^2$  flops. The total is  $\sum_j j^2 \approx p^3/3$ . Each leverage is a triangular solve with  $\mathbf{R}^\top$  ( $p^2$  flops) and a squared norm. The projection  $\mathbf{M}$  has  $n^2$  entries, which for  $n = 10^6$  is too many to store, and only its diagonal is ever needed.

**Exercise B2.**

Two groups of observations have sizes  $n_a, n_b$ , means  $\mathbf{m}_a, \mathbf{m}_b$  and centred cross-product matrices  $\mathbf{C}_a, \mathbf{C}_b$ . Show that the pooled data have centred cross-product matrix

$$\mathbf{C} = \mathbf{C}_a + \mathbf{C}_b + \frac{n_a n_b}{n_a + n_b} (\mathbf{m}_a - \mathbf{m}_b)(\mathbf{m}_a - \mathbf{m}_b)^\top.$$

Show that [Proposition 10.2.4](#) is the special case  $n_b = 1$ , and explain how the formula allows cross-products to be computed in parallel on separate blocks of data.

### 10.2.6.3 C. Going deeper

#### Exercise C1.

Let  $\mathbf{A}$  be nonnegative definite of rank  $r$ . Show that the Cholesky recursion with *symmetric pivoting*, which at step  $j$  moves the largest remaining diagonal entry of the current Schur complement to position  $j$ , produces a permutation  $\mathbf{\Pi}$  and  $\mathbf{\Pi}^\top \mathbf{A} \mathbf{\Pi} = \mathbf{R}^\top \mathbf{R}$  with  $\mathbf{R} = \begin{pmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}$  and  $\mathbf{R}_{11}$  an  $r \times r$  nonsingular upper triangle. (Use Proposition 1.7.5(a).) How is this related to the pivoted QR of [Section 10.7](#)?